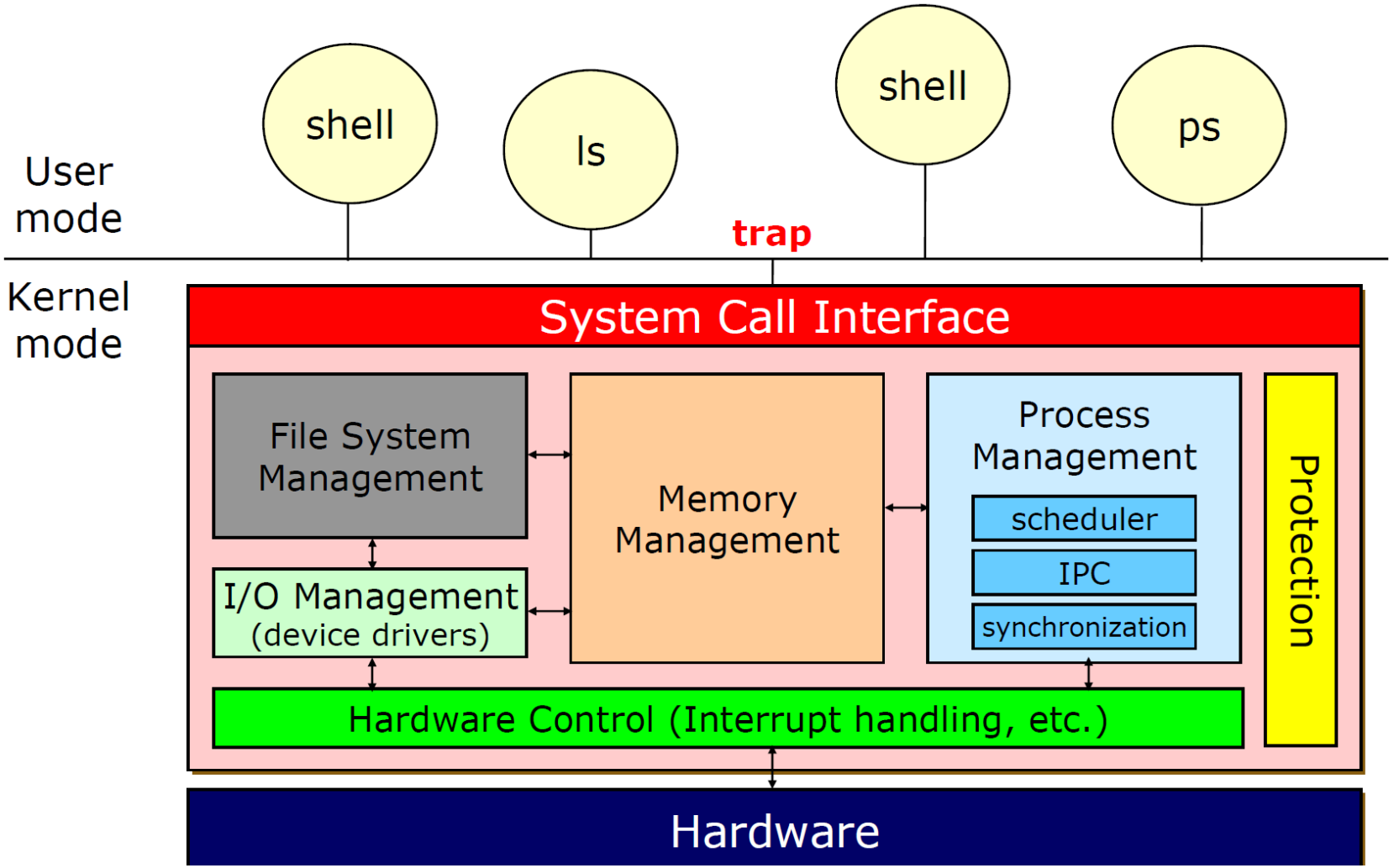
CS330 Operating Systems and Lab.

I/O Systems

Spring 2026

Instructor : Insik Shin

Operating Systems Structure



I/O Subsystem

- I/O subsystem is a major concern of OS design
  - Two conflicting trends with I/O device technology
    - Increasing standardization of s/w and h/w interfaces
      - Good to incorporate advanced device generations
    - Increasingly broad variety of I/O devices
      - Imposing difficulties: some new devices are unlike previous ones

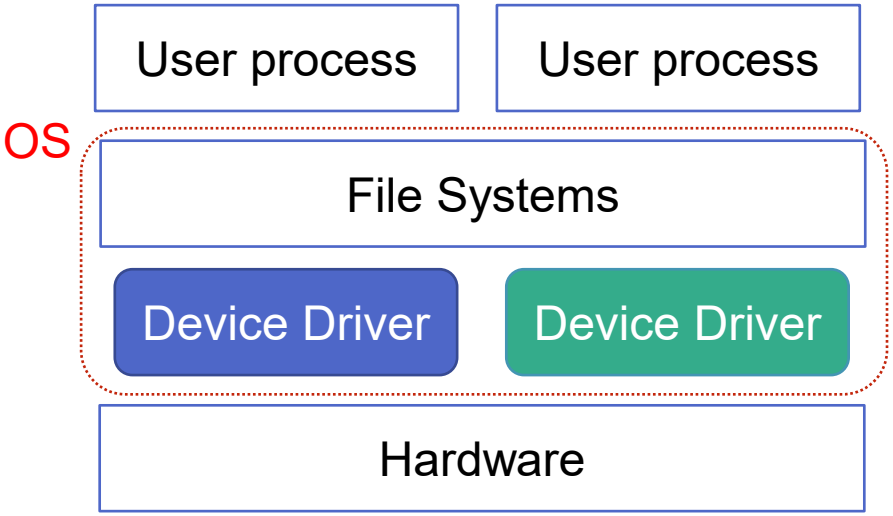
I/O Hardware

- Incredible variety of I/O devices
  - Storage devices: disks, tapes, SSDs
  - Transmission devices: network cards, modems
  - Human-interface devices: screen, keyboard, mouse
  - Sensors in mobile devices: camera, GPS, accelerometer, gyro, temperature, blood pressure

  - Despite of variety, we need only a few concepts to understand how the devices are attached and how S/W can control H/W

I/O Subsystem

- Approaches
  - Basic I/O H/W accommodate a wide variety of devices
    - *Ports, buses, device controllers*
  - S/W: OS is structured to use *device-driver* modules
    - device drivers present a uniform device access interface to I/O subsystem, like system calls btw OS and applications

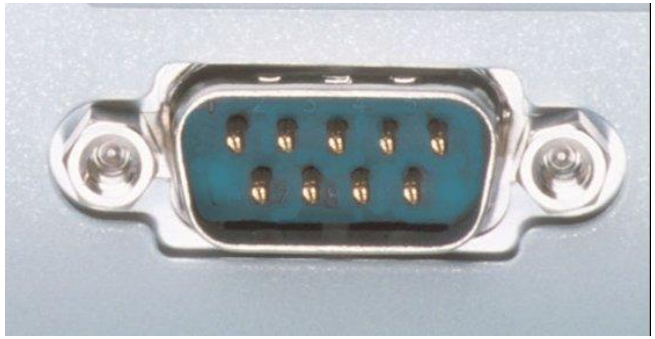


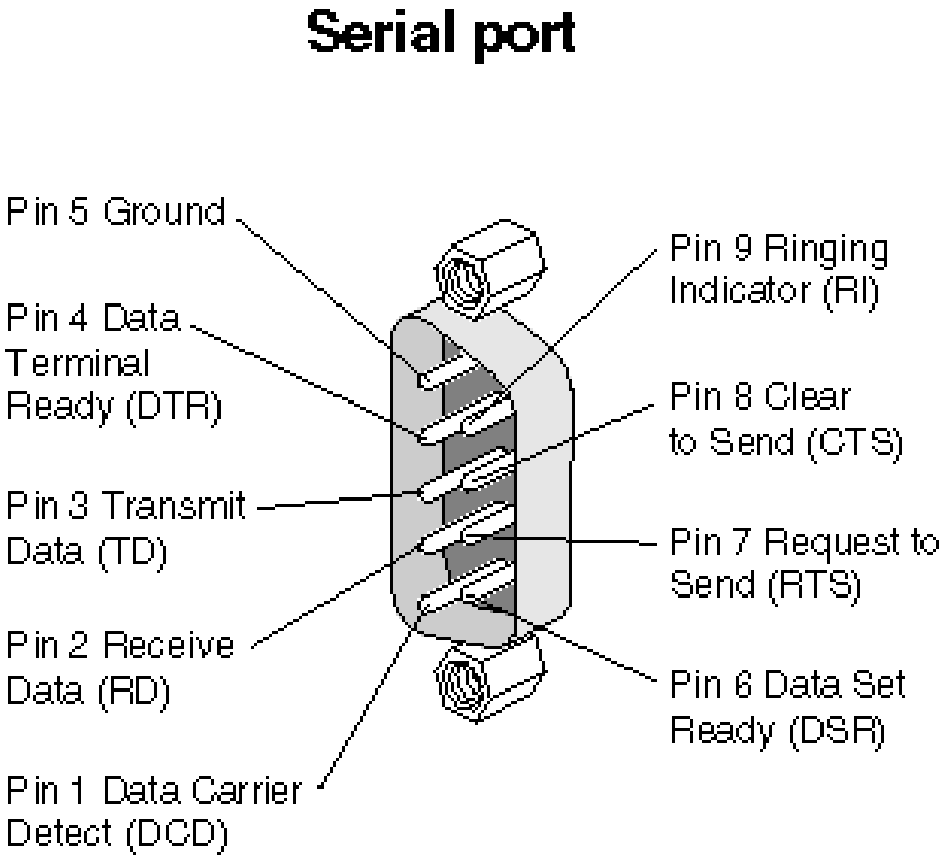
I/O Hardware - Common concepts

- Port
  - Connection point
- Bus (daisy chain or shared direct access)
  - A set of wires
  - A protocol specifying messages sent on the wires
  - *PCI* bus (the common PC system bus)
- Controller (host adapter)
  - A collection of electronics that operates a port, a bus, or a device

Serial Port & Wire

- Port
  - Connection point



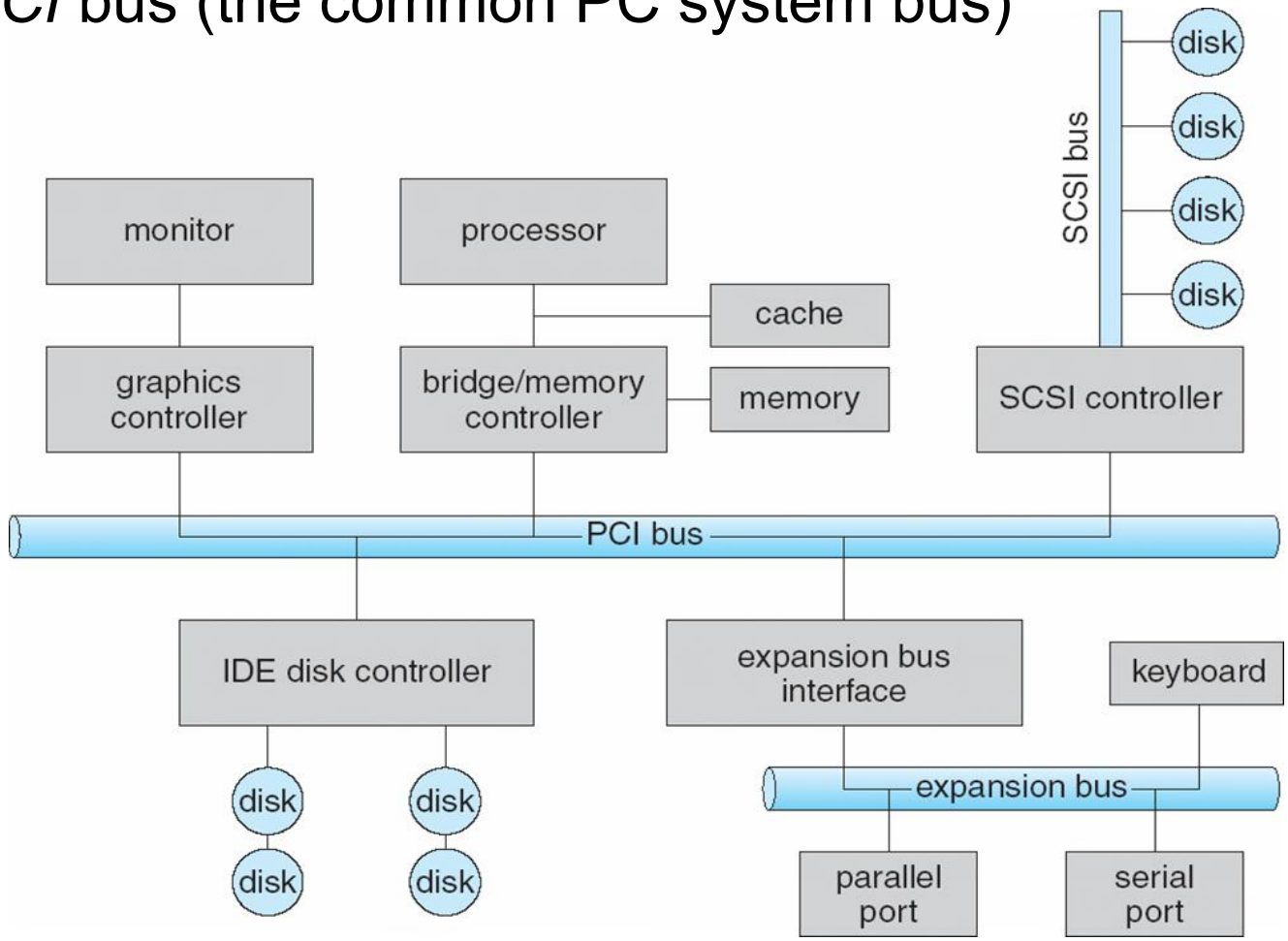


I/O Hardware - Common concepts

- Port
  - Connection point
- Bus (daisy chain or shared direct access)
  - A set of wires
  - A protocol specifying messages sent on the wires
  - *PCI* bus (the common PC system bus)
- Controller (host adapter)
  - A collection of electronics that operates a port, a bus, or a device

A Typical PC Bus Structure

- Bus
  - *PCI* bus (the common PC system bus)

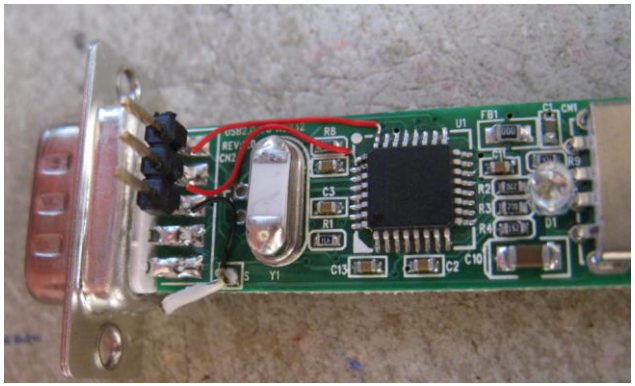


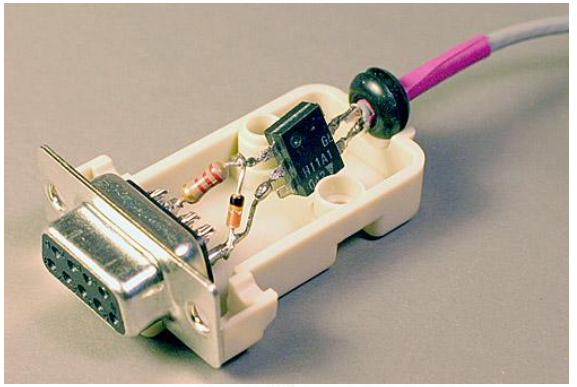
I/O Hardware - Common concepts

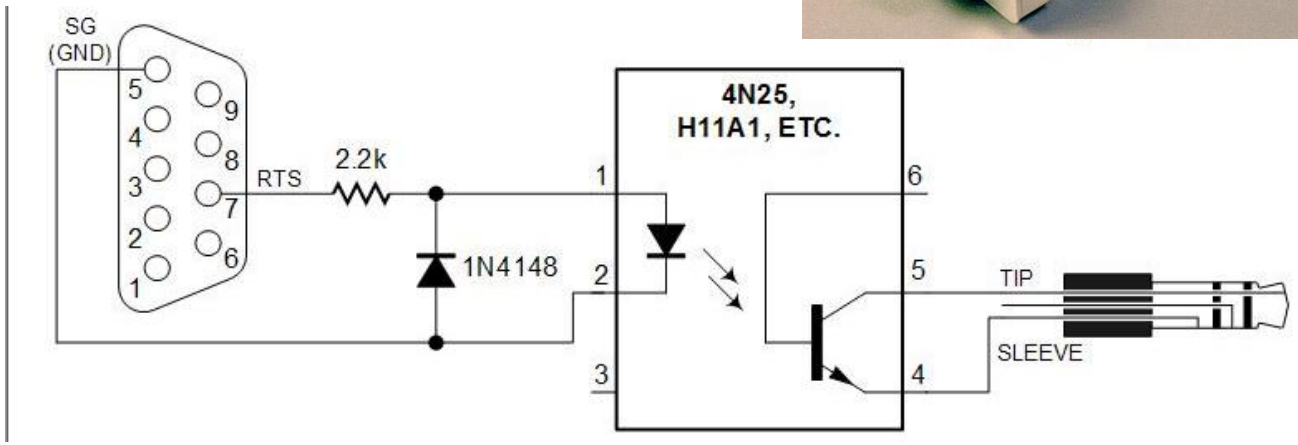
- Port
  - Connection point
- Bus (daisy chain or shared direct access)
  - A set of wires
  - A protocol specifying messages sent on the wires
  - *PCI* bus (the common PC system bus)
- Controller (host adapter)
  - A collection of electronics that operates a port, a bus, or a device

I/O Hardware – Controller Examples

- Controller examples
  - A serial-port controller (simple)
    - A single chip (or its portion) in computer that controls the signals on the wires of a serial port

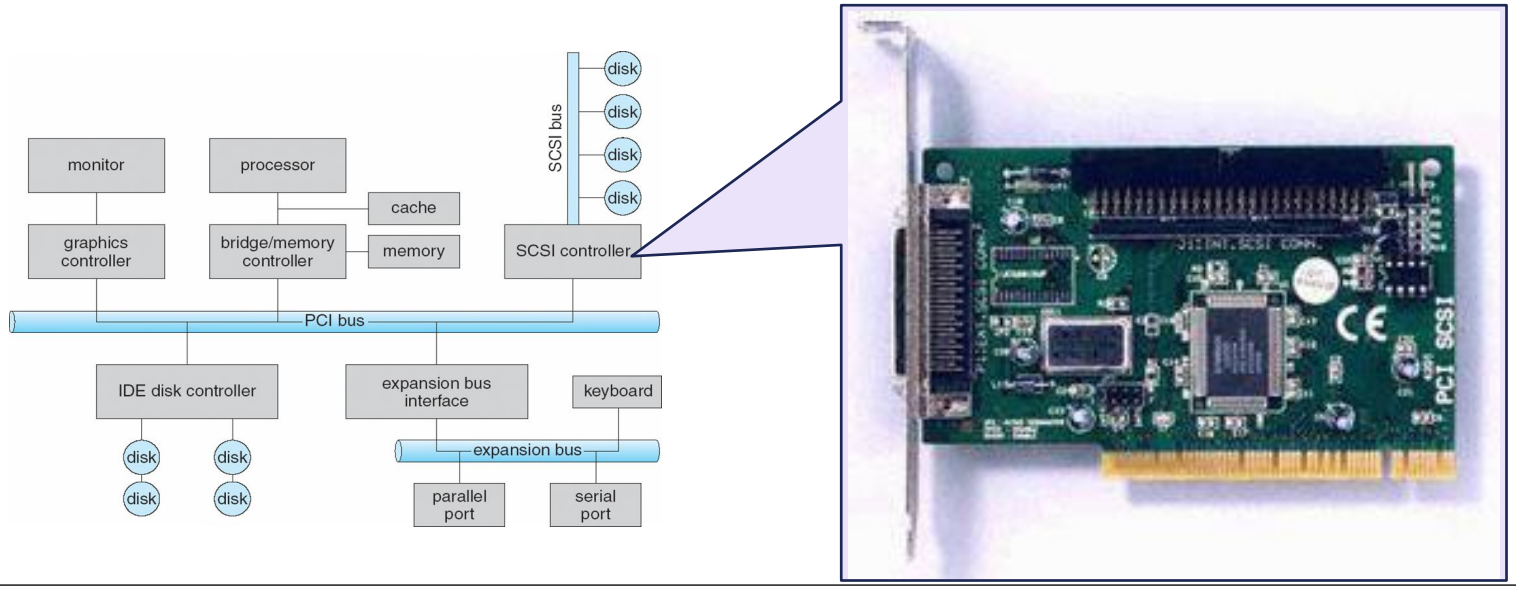






I/O Hardware – Controller Examples

- Controller examples
  - A SCSI bus controller (complex)
    - A separate circuit board that plugs into computer
    - Contains a processor, microcode, private memory to process the SCSI protocol messages
    - Disk can have its own built-in (disk) controller that implements the disk side of the SCSI protocol, as well as many disk tasks (bad-sector mapping, prefetching, buffering, caching)

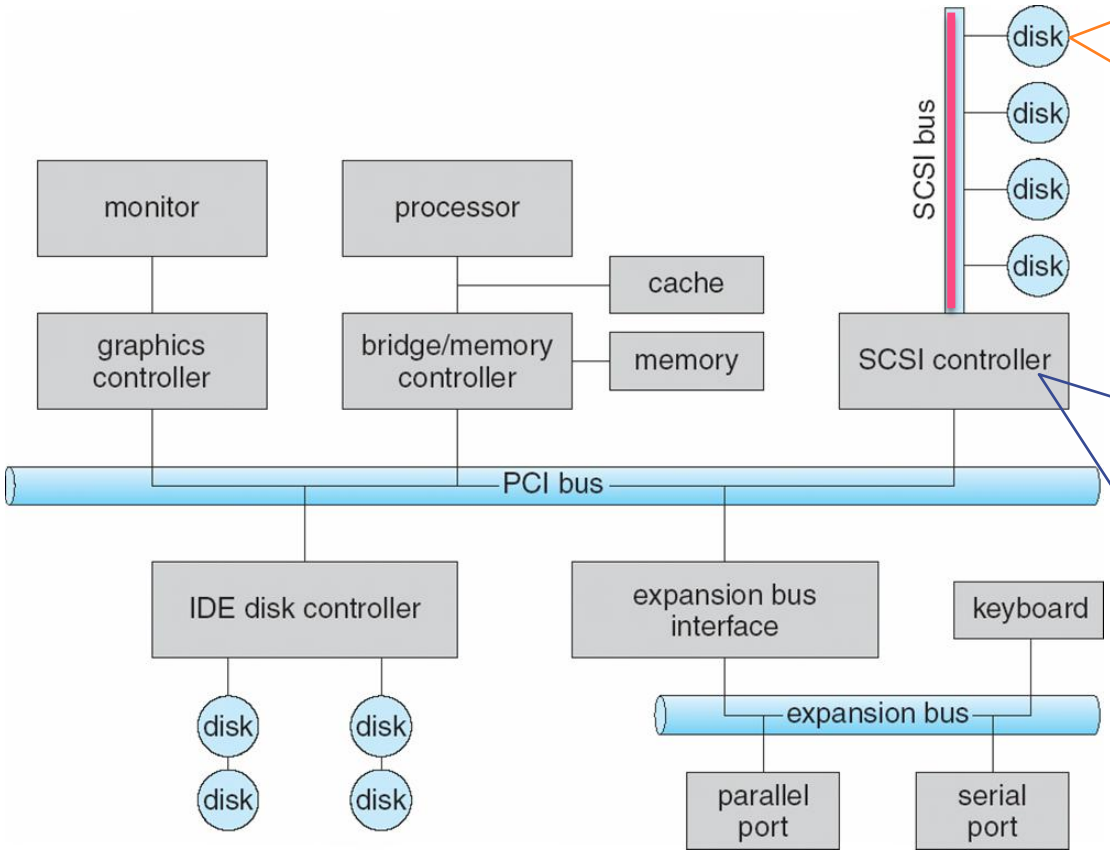


I/O Hardware – Controller Examples

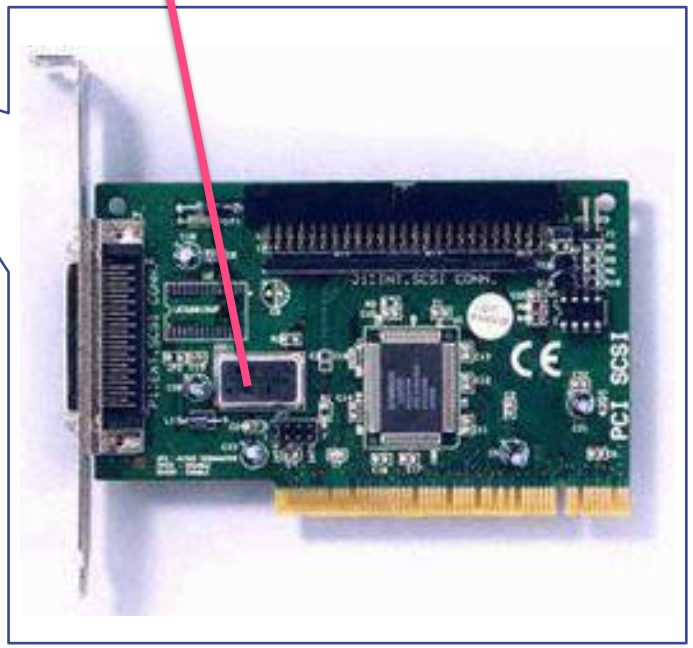
- Controller examples
  - A hard disk controller (complex)
    - Disk can have its own built-in (disk) controller that implements the disk side of the SCSI protocol, as well as many disk tasks (bad-sector mapping, prefetching, buffering, caching)

The TrueStore chip set from Agere Systems comprises a read-channel preamplifier, a motor controller, and a hard-disk controller.

SCSI Bus Connection



SCSI Bus



I/O Hardware

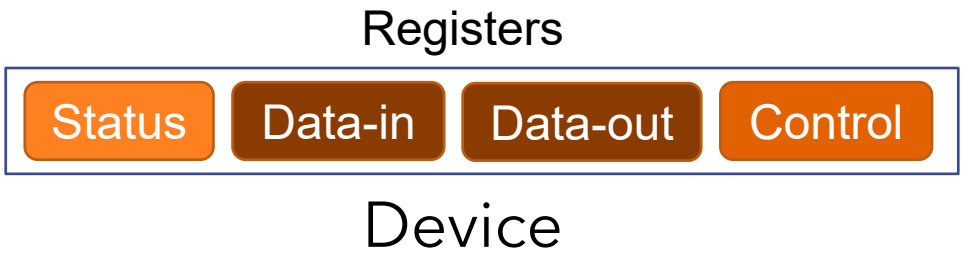
- Communication btw CPU & devices
  - Controller has registers for data and control signals
  - CPU reads/writes bit patterns in these registers
- Devices have addresses, used by
  - Direct I/O instructions
    - Via I/O port address, bus lines, device registers
  - Memory-mapped I/O
    - Device registers are mapped into address space of the CPU
    - Example: graphics controller
    - Transfer of millions of byte: faster to write them directly into memory than to issue millions of I/O instructions for

Device I/O Port Locations on PCs (partial)

I/O address range (hexadecimal)	device
000–00F	DMA controller
020–021	interrupt controller
040–043	timer
200–20F	game controller
2F8–2FF	serial port (secondary)
320–32F	hard-disk controller
378–37F	parallel port
3D0–3DF	graphics controller
3F0–3F7	diskette-drive controller
3F8–3FF	serial port (primary)

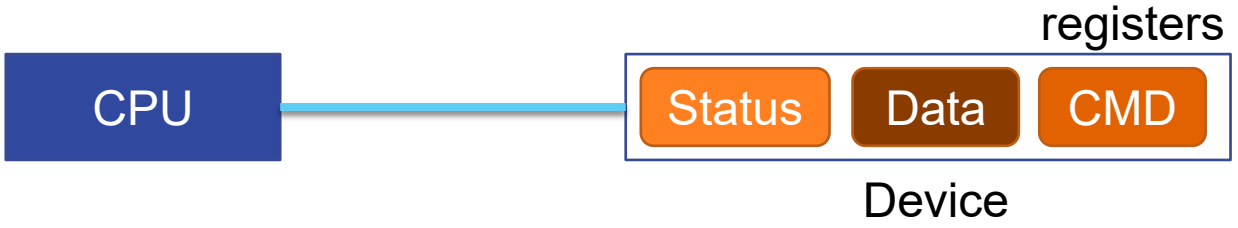
I/O Port Registers

- Four categories for I/O port registers
  - Data-in registers
  - Data-out registers
  - Status registers
    - Host reads status, such as whether the current command has completed, whether a byte is available to be read, whether an error has occurred
  - Control registers
    - Host writes to start a command or to change the mode of a device



Communication btw Host & Controller

- Handshaking communication
  - Complete protocol for interaction between host and controller is intricate, but its basic *handshaking* notion is simple
  - Host ’s writing a byte through a port
    1. Host repeatedly reads “busy” bit until it becomes clear
    2. Host sets “write” bit in the “command” register and writes a byte into “data-out” register
    3. Host sets “command-ready” bit
    4. When controller notices “command-ready” bit, it sets “busy” bit
    5. Controller reads “command” register, reads “data-out” register to get the byte, and does I/O to the device
    6. Controller clears “command-ready” bit, “error” bit (indicating successful I/O operation), and “busy” bit

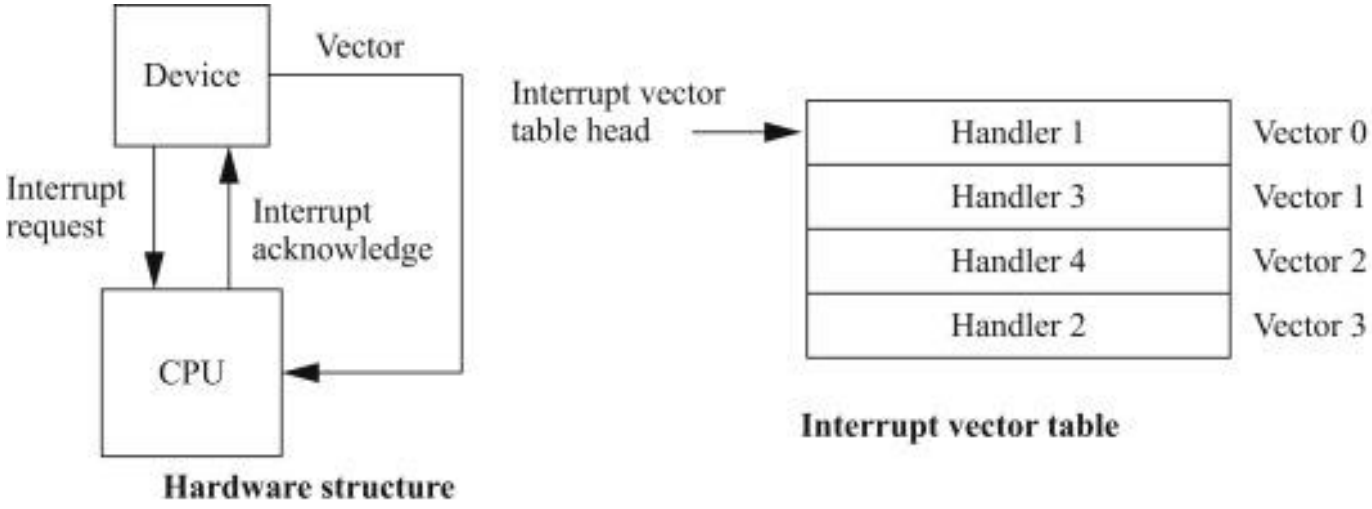


Polling

- The previous loop is repeated for each byte
- In step 1, host is *busy-waiting* or *polling*
  - The basic polling operation is efficient
    - Can be done only with three CPU-instruction cycles
  - But polling becomes inefficient when it rarely finds a device to be ready
    - Then it may be more efficient to arrange for the controller to notify CPU when it becomes ready

Interrupts

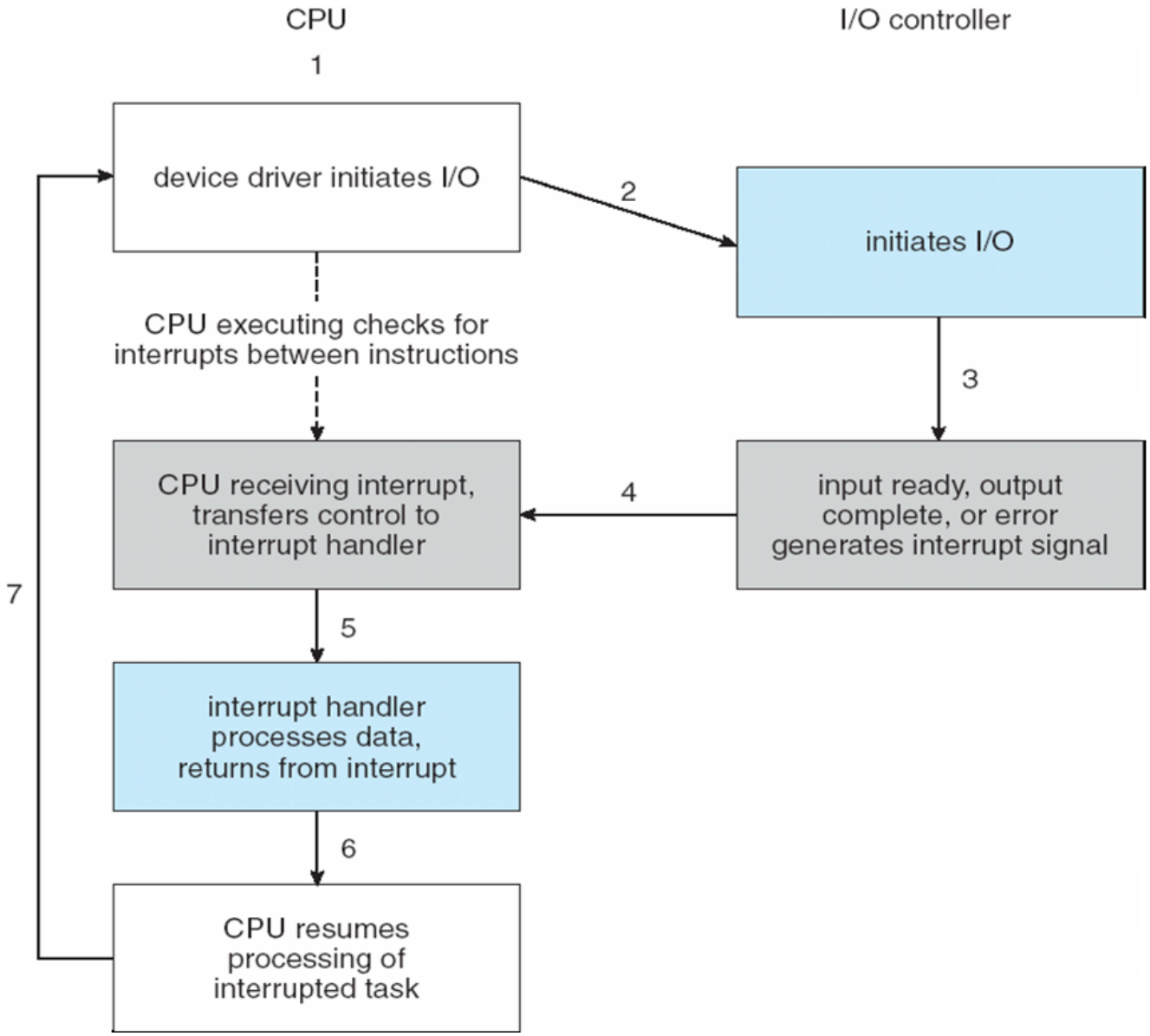
- Hardware mechanism for interrupt
  - CPU has *interrupt-request line* that it senses after executing every instruction
  - I/O device controller *raises* an interrupt by asserting a signal on the line
  - CPU *catches* the interrupt and *dispatches* it to the *interrupt handler*
  - Handler *clears* the interrupt by servicing the device



Interrupts

- Hardware mechanism for interrupt
  - CPU has *interrupt-request line* that it senses after executing every instruction
  - I/O device controller *raises* an interrupt by asserting a signal on the line
  - CPU *catches* the interrupt and *dispatches* it to the *interrupt handler*
  - Handler *clears* the interrupt by servicing the device
- How to service interrupts: immediately or later
  - CPU often has two interrupt request lines
  - Non-maskable for urgent interrupts
  - maskable to ignore or delay some interrupts (I/O devices) to do some critical processing without any interrupts

Interrupt-Driven I/O Cycle



Intel Pentium Processor Event-Vector Table

vector number	description
0	divide error
1	debug exception
2	null interrupt
3	breakpoint
4	INTO-detected overflow
5	bound range exception
6	invalid opcode
7	device not available
8	double fault
9	coprocessor segment overrun (reserved)
10	invalid task state segment
11	segment not present
12	stack fault
13	general protection
14	page fault
15	(Intel reserved, do not use)
16	floating-point error
17	alignment check
18	machine check
19–31	(Intel reserved, do not use)
32–255	maskable interrupts

Direct Memory Access

- Programmed I/O (PIO)
  - Task of watching status bits and feeding data into control register one byte at a time
  - Wasteful for an expensive general-purpose CPU to do PIO for large data movement, i.e., with a disk drive
- DMA Controller (Direct-Memory-Access)
  - Special-purpose processor that transfers data directly between memory and I/O device bypassing CPU
  - It offloads the burden of CPU to do PIO for large transfer

Six Step Process to Perform DMA Transfer